

CXL-SDK: Building SDK for CXL Shared Memory

Submission #31

Abstract

Memory interconnect technologies such as Compute Express Link (CXL) and Unified Bus (UB) enable multiple machines to share memory and communicate through load/store instructions. This creates opportunities for systems such as databases, RPC frameworks, and data analytics engines to reduce data movement and coordination overhead. However, programming these platforms remains difficult for two reasons: hardware platforms expose different features and synchronization semantics, and software lacks a unified development kit that provides reusable interfaces for shared-memory programming. This paper argues for a software development kit (SDK) for CXL-like memory interconnects. We next show what such an SDK should include and how its components can hide hardware complexity from applications and system libraries.

1 Introduction

Memory interconnect technologies such as Compute Express Link (CXL) [1] and Unified Bus (UB) [3] let multiple machines share the same physical memory and access it with load/store primitives. This changes the data path of distributed systems: shared state can be updated in place through memory—with loads, stores, and atomics—rather than serialized into messages and transferred through a network stack.

Recent systems show that building systems on top of CXL/UB shared memory can produce large gains. CXL-aware RPC like CXL-SHM [22], HydraRPC [12], and RPCool [13] place request and response buffers in shared memory and introduce bypass-by-reference message passing to avoid serialization and memory copies. Database systems like Tigon [9] and PolarDB-CXL [20] place cross-partition database state in CXL and synchronize transactions through shared-memory primitives, avoiding the message exchanges of distributed commit protocols. Analytics systems like Spark [21] and Ray [14] can place shuffle data in shared memory to avoid serialization and memory copies [2]. In addition, LLM systems like Beluga [19] place the KVCache in CXL and allow rack-scale machines to share KVCache efficiently.

However, today’s systems are largely *one-off efforts*: each rebuilds its own low-level mechanisms around a particular hardware model. Hardware models differ in memory sharing patterns and cache coherence assumptions. Memory sharing patterns differ in how shared memory is owned and accessed. In one model, memory forms an independent shared pool that is not owned by any particular machine (e.g., CXL). In another model, each machine retains ownership of its local

Table 1: Different hardware cache-coherence assumptions in CXL/UB-like shared memory.

Model	Data access	Example
Full CC	cached by all hosts	CXL 3.0 [1]
Small-region CC	coherent only in a small region	AMD [11], Tigon [9]
No CC + atomics	not coherent, w/ global atomics	CXLAlloc [15], Huawei UB [3], Beluga [19], PolarDB-CXL [20]

memory but exposes part of it for remote access by other machines (e.g., UB). Cache coherence assumptions differ because supporting global cache coherence across all machines is not practical at scale, and different vendors and academic studies target different weaker cache coherence assumptions. As summarized in Table 1, weaker cache-coherence assumptions include: (1) coherence for only a small memory region, e.g., several hundred MiB; (2) no cache coherence across machines but with cross-host atomics for synchronization.

We argue that a software development kit (SDK) for CXL/UB shared memory is the right approach to enable a broader range of workloads to benefit from CXL/UB. Such an SDK should (1) provide reusable components, and (2) hide hardware differences from applications. Our core position is that *the data structure layer is the right abstraction boundary* for this SDK. Many commonly used system libraries—object stores, file systems, and RPC frameworks—are ultimately built on top of data structures, making this layer a natural convergence point where hardware complexity can be hidden without exposing it to higher layers.

In the following sections, we will describe each layer of our proposed CXL-SDK (§2), detail the advantages of using data structure as the core (§3), and then summarize our current implementation and further roadmaps (§4).

2 Overview of CXL-SDK

Figure 1 shows the layered architecture of our proposed CXL-SDK. We then describe each layer from top to bottom.

Application layer lists applications that can benefit from CXL-SDK.

Library layer includes object stores, file systems, and RPC frameworks, which are directly used by applications. We select these three based on a survey of common datacenter workloads: as shown in Table 2, analytics frameworks (Spark/Flink/Ray) need file systems or object stores for data shuffle, microservices need RPC, and LLM inference and recommendation systems need object stores for KVCache and embedding tables.

Data structure layer exposes CXL-aware data structures for libraries and applications. Libraries are built on top of

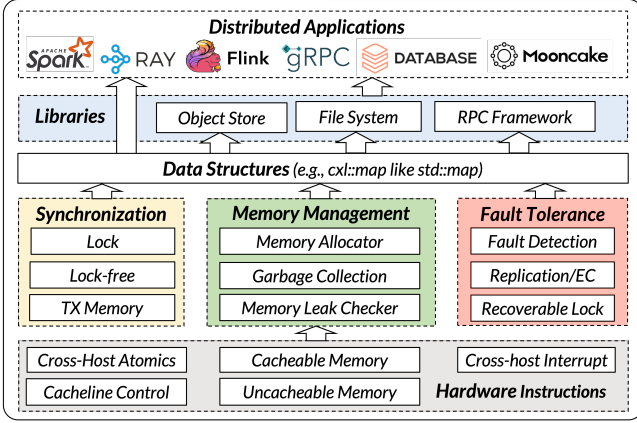


Figure 1: CXL-SDK Architecture.

data structures: file systems use tree-based data structures, object stores use hash- or tree-based data structures, and RPC frameworks are built on top of queues.

In addition, this layer exposes STL-style APIs such as maps, sets, lists, queues, hash tables, and tree indexes (e.g., `cxl::map` like `std::map` in C++), since STL is the most familiar data structure interface for C++ programmers and minimizes the adoption cost of CXL-SDK. For example, analytics frameworks can share intermediate results through vectors or hash tables; graph processing can share graph state through graph-shaped containers; databases can share records through tree indexes. These data structures run correctly on different hardware models, and hide the underlying hardware complexity.

Mechanism layer includes mechanisms like synchronization, memory management and fault tolerance, which are the foundation for building higher-level layers.

- **Synchronization** is responsible for providing synchronization primitives on different cache-coherence assumptions. It provides basic synchronization primitives like locks, lock-free protocols, and transactional memory. For example, on a platform with small-region cache coherence, locks should be implemented using atomic variables stored in cache-coherent regions; on a platform with no cache coherence but cross-host atomics, locks should be implemented using cross-host atomics.
- **Memory management** is responsible for providing memory allocation, garbage collection, and memory leak checking. Existing DRAM and PM allocators [4–6, 8, 10, 18], garbage collection mechanisms [7, 17], and auxiliary debugging tools (e.g., AddressSanitizer [16]) do not operate correctly on platforms with weaker cache-coherence models, necessitating new CXL/UB-aware implementations.
- **Fault tolerance** provides fault detection and recovery mechanisms to handle CXL/UB’s partial failure problem [22], where machines and their used memory may

Table 2: Apps and their needed SDK components.

Application scenario	SDK infrastructure
Spark/Ray/Flink	file system, object store
Microservices	zero-copy RPC
Databases	tree/hash data structures
LLM inference/RecSys	object store, tree/hash data structures

fail independently. CXL-SDK provides (1) fault detection mechanisms like Lupin’s fabric-based heartbeat checking [23], (2) replication/erasure coding for memory backup and recovery, and (3) recoverable locks [23] for handling the partial failure problem.

Hardware instruction layer provides a uniform hardware instruction interface by wrapping basic primitives like cross-host atomics, cacheline control, and cross-host interrupts on different platforms (Intel+CXL, ARM+UB).

3 Benefits of a Unified Data Structure Layer

Having applications and libraries built on a unified set of SDK-managed data structures brings two concrete benefits. First, it gives memory management more semantics for identifying reachable live data under partial failure. Partial failures can leak memory when a process allocates an object and fails before linking it into a shared structure [22]. If applications allocate through SDK data structures, each data structure’s root (e.g., tree’s root node, list’s head node) defines the reachable live data; unreachable allocations can be checked and reclaimed by asynchronous garbage collection.

Second, it enables upper-level libraries to use data structure’s semantics for coordination. Take the RPC framework as an example: CXL RPC frameworks can share a data structure by passing a reference (e.g., pointer of the root node) instead of serializing and copying the entire data structure [13, 22]. If the RPC framework knows that an argument is an SDK-managed STL-style data structure, the sender and receiver can share a consistent view of its memory layout. The receiver can therefore directly access and modify the data structure in shared memory.

4 Current Status and Roadmap

We have implemented the hardware instruction layer, the mechanism layer, and the data structure layer (list, queue, hash table, tree index, etc.). Ongoing work includes: (1) implementing the library layer (file systems, object stores, and RPC frameworks) on top of CXL-aware data structures; (2) completing support for additional hardware platforms so the SDK runs portably across different CXL/UB configurations; and (3) adding remaining features such as replication, erasure coding, and memory leak checker. We will release the SDK in stages and invite the community to contribute.

References

- [1] 2024. *CXL Consortium. Compute Express Link Specification, Revision 3.1, 2023*.
- [2] 2025. *CXL Use Case: Ray*.
- [3] 2025. *Unified Bus: An Interconnect Protocol for SuperPoD*.
- [4] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. 2000. Hoard: A Scalable Memory Allocator for Multithreaded Applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*. Association for Computing Machinery, 117–128. doi:10.1145/378993.379232
- [5] Kumud Bhandari, Dhruva R. Chakrabarti, and Hans-J. Boehm. 2016. Makalu: Fast Recoverable Allocation of Non-Volatile Memory. *SIGPLAN Not.* 51, 10 (October 2016), 677–694.
- [6] Jason Evans. 2006. A Scalable Concurrent malloc(3) Implementation for FreeBSD. In *Proceedings of the BSDCan Conference*. Ottawa, Canada.
- [7] Keir Fraser. 2004. *Practical Lock-Freedom*. Ph.D. Dissertation. University of Cambridge. Technical Report UCAM-CL-TR-579.
- [8] Sanjay Ghemawat and Paul Menage. 2007. *TCMalloc: Thread-Caching Malloc*. Technical Report. Google. <https://goog-perftools.sourceforge.net/doc/tcmalloc.html>
- [9] Yibo Huang, Haowei Chen, Newton Ni, Yan Sun, Vijay Chidambaram, Dixin Tang, and Emmett Witchel. 2025. Tigon: a distributed database for a CXL pod. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation (Boston, MA, USA) (OSDI '25)*. USENIX Association, USA, Article 7, 20 pages.
- [10] Intel Corporation. 2020. *PMDK: Persistent Memory Development Kit*.
- [11] Sunita Jain, Nagaradhesh Yeleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory Sharing with CXL: Hardware and Software Design Approaches. arXiv:2404.03245 [cs.ET] <https://arxiv.org/abs/2404.03245>
- [12] Teng Ma, Zheng Liu, Chengkun Wei, Jialiang Huang, Youwei Zhuo, Haoyu Li, Ning Zhang, Yijin Guan, Dimin Niu, Mingxing Zhang, and Tao Ma. 2024. HydraRPC: RPC in the CXL Era. In *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, Santa Clara, CA, 387–395. <https://www.usenix.org/conference/atc24/presentation/ma>
- [13] Suyash Mahar, Ehsan Hajjajisani, Seungjin Lee, Zifeng Zhang, Mingyao Shen, and Steven Swanson. 2024. Telepathic Datacenters: Fast RPCs using Shared CXL Memory. arXiv:2408.11325 [cs.DC] <https://doi.org/10.48550/arXiv.2408.11325>
- [14] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 561–577. <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [15] Newton Ni, Yan Sun, Zhiting Zhu, and Emmett Witchel. 2026. Cxlalloc: Safe and Efficient Memory Allocation for a CXL Pod. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 528–545. doi:10.1145/3779212.3790149
- [16] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC '12)*. USENIX Association, 309–318.
- [17] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (Farmington, Pennsylvania) (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 18–32. doi:10.1145/2517349.2522713
- [18] Haris Volos, Andres Jaan Tack, and Michael M. Swift. 2011. Mnemosyne: Lightweight Persistent Memory. In *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVI)*. Association for Computing Machinery, 91–104.
- [19] Xinjun Yang, Qingda Hu, Junru Li, Feifei Li, Yicong Zhu, Yuqi Zhou, Qiuru Lin, Jian Dai, Yang Kong, Jiayu Zhang, Guoqiang Xu, and Qiang Liu. 2026. Beluga: A CXL-Based Memory Architecture for Scalable and Efficient LLM KVCache Management. *Proc. ACM Manag. Data* 4, 1, Article 13 (April 2026), 29 pages. doi:10.1145/3786627
- [20] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Gerry Fan, Yang Kong, Bo Wang, Jing Fang, Yuhui Wang, Tao Huang, Wenpu Hu, Jim Kao, and Jianping Jiang. 2025. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Companion of the 2025 International Conference on Management of Data (Berlin, Germany) (SIGMOD/PODS '25)*. Association for Computing Machinery, New York, NY, USA, 689–702. doi:10.1145/3722212.3724460
- [21] M. Zaharia, Mosharaf Chowdhury, Michael J. Franklin, S. Shenker, and Ion Stoica. 2010. Spark: Cluster Computing with Working Sets. In *USENIX Workshop on Hot Topics in Cloud Computing*. 10–10.
- [22] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674. doi:10.1145/3600006.3613135
- [23] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. 2024. Lupin: Tolerating Partial Failures in a CXL Pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems (Austin, TX, USA) (DIMES '24)*. Association for Computing Machinery, New York, NY, USA, 41–50. doi:10.1145/3698783.3699377